



TRABALLO FIN DE GRAO
GRAO EN ENXEÑARÍA INFORMÁTICA
MENCIÓN EN COMPUTACIÓN



Diseño e implementación de un sistema de navegación volumétrica para agentes voladores en videojuegos

Estudiante: Javier Manotas Ruiz
Dirección: Enrique Fernández Blanco
Alejandro Puente Castro

A Coruña, xuño de 2026.

Índice general

1	Introducción	1
1.1	Motivación	2
1.2	Objetivos	4
1.3	Estructura de la memoria	5
2	Fundamentos	6
2.1	Fundamentos tecnológicos	6
2.1.1	Unity y su modelo de componentes	6
2.1.2	Herramientas proporcionadas por el editor	7
2.1.3	Motor de físicas y consultas espaciales	8
2.1.4	Data-Oriented Technology Stack	8
2.1.5	Características de bajo nivel en C#	9
2.2	Fundamentos teóricos	10
2.2.1	Búsqueda de caminos	10
2.2.2	Planificación de caminos	14
2.2.3	Integridad estructural mediante algoritmos Hash	15
3	Estado de la cuestión	16
3.1	Herramientas disponibles	16
3.1.1	Herramientas genéricas	16
3.1.2	Soluciones específicas de motor	17
3.2	Comparativa de mercado	19
3.2.1	Justificación del entorno de desarrollo	19
3.3	DAFO	20
3.3.1	Análisis interno	20
3.3.2	Análisis externo	20
3.3.3	Conclusiones	21

Lista de acrónimos	22
Glosario	23
Bibliografía	25

Índice de figuras

1.1	Ejemplo de malla 2.5D generada por NavMesh sobre un entorno tridimensional.	2
1.2	Visualización de los 6 grados de libertad.	3
2.1	Interfaz del editor de Unity. A la izquierda (en rojo) jerarquía de <code>GameObject</code> y a la derecha (en azul) inspector de componentes de un objeto.	7
2.2	Comparativa de POO y DOD.	9
2.3	Versión 2D de los códigos de Morton.	12
2.4	Ejemplo curva Catmull Rom interpolando los puntos de control.	13
3.1	Visualización de navegación por cualquier superficie.	17

Índice de tablas

3.1	Resumen de las alternativas existentes	19
3.2	Matriz DAFO del proyecto	21

Introducción

EL desarrollo de entornos virtuales contemporáneos, como videojuegos y simulaciones interactivas, exige dotar a sus agentes de la inteligencia suficiente para interactuar coherentemente con su entorno. El comportamiento de estas entidades resulta clave para garantizar la credibilidad, funcionalidad y realismo del entorno virtual, especialmente en escenarios complejos y dinámicos. A pesar de que se trate de un problema que lleva décadas siendo explorado en el campo de la inteligencia artificial [1], a día de hoy, dista de ser trivial cuando se persigue un comportamiento robusto y eficiente.

Dentro de este ámbito, un requisito fundamental es la **navegación autónoma**. Este proceso permite a un agente desplazarse desde un punto de origen hasta un destino específico calculando una ruta viable y esquivando, de manera eficiente, la geometría y los obstáculos presentes en la escena. En el contexto de videojuegos, este comportamiento recae habitualmente en los personajes no jugables, conocidos en inglés como **Non-Playable Character (NPC)**. Su capacidad para desenvolverse de forma autónoma condiciona en gran medida la percepción de inteligencia del sistema. Abordar este reto no solo implica determinar el camino más corto, sino también garantizar que el movimiento resultante sea compatible con las restricciones físicas del entorno y con el comportamiento esperado del agente [2].

El problema de la búsqueda de rutas, conocido en la literatura como *pathfinding*, constituye uno de los casos de estudio más explorados en el campo de la inteligencia artificial aplicada a entornos virtuales [3]. En la actualidad, se considera un problema ampliamente resuelto para agentes terrestres en escenarios estáticos [4], hasta el punto de que los principales motores de desarrollo, tanto comerciales como de código abierto, integran soluciones nativas y robustas para este fin. Ejemplos representativos de ello son el Navigation System en Unreal Engine [5], el sistema nativo de NavMesh en Unity [6] o el componente NavigationServer3D en Godot [7], los cuales permiten implementar comportamientos de navegación complejos con un coste computacional asumible.

La gran mayoría de estas herramientas se fundamentan en la biblioteca de código abierto Recast Navigation [8]. Este sistema permite generar una **mallá de navegación**, que recibe el nombre de NavMesh, y que representa una **aproximación 2.5D** del entorno transitable. Aunque el escenario se modela en tres dimensiones, el cálculo de rutas se proyecta sobre superficies esencialmente bidimensionales que definen las zonas donde los agentes pueden apoyarse o caminar, tal y como puede apreciarse en la Figura 1.1. A partir de esta geometría, se construye un grafo sobre el cual se aplican algoritmos de búsqueda heurística, siendo el algoritmo A^* el caso estándar de aplicación [9].

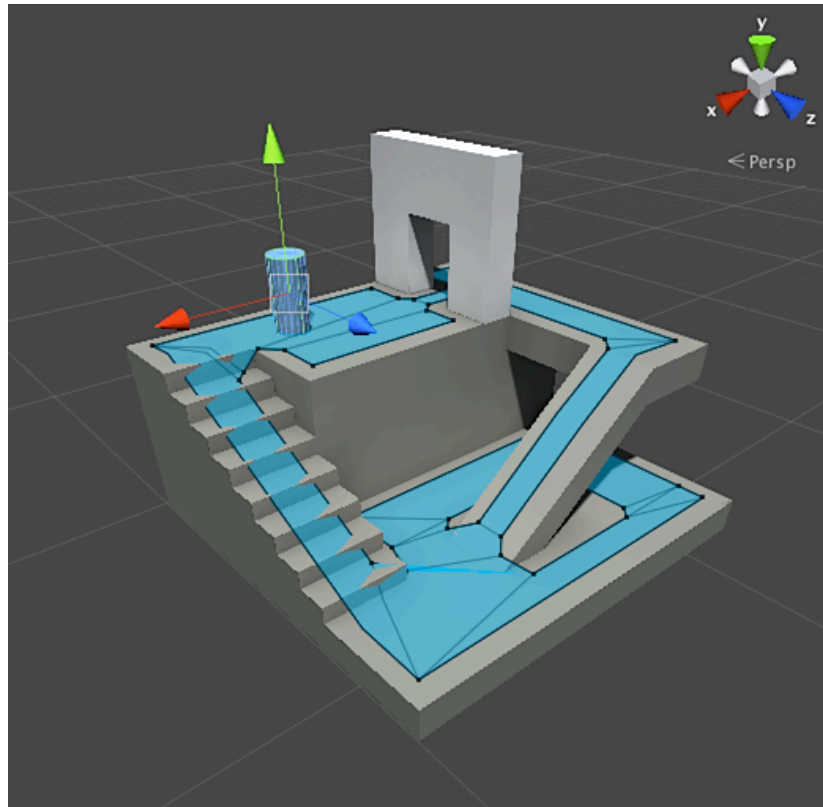


Figura 1.1: Ejemplo de mallá 2.5D generada por NavMesh sobre un entorno tridimensional. Imagen extraída de [10].

1.1 Motivación

La necesidad de crear una herramienta que dote a agentes voladores de la capacidad de moverse a través de un espacio plenamente tridimensional surge del limitado soporte que ofrecen las tecnologías dominantes en el desarrollo de videojuegos actual. Las soluciones tradicionales basadas en mallás 2.5D resultan ideales para personajes terrestres, ya que presuponen que el agente está anclado al suelo y que su movimiento puede reducirse a un plano horizontal cuya

altura es una propiedad derivada del terreno. Sin embargo, este supuesto no se cumple para entidades con capacidad de vuelo, tales como drones, naves espaciales, enjambres de insectos o criaturas submarinas como peces, las cuales requieren operar con **seis grados de libertad**: tres traslaciones a lo largo de los ejes X , Y y Z y tres rotaciones independientes alrededor de esos mismos ejes, conocidas como *pitch*, *yaw* y *roll*, tal y como se ilustra en la Figura 1.2. Ante la posibilidad de que los agentes puedan modificar su trayectoria en cualquier dirección y altura, un plano transitable resulta insuficiente y se hace imprescindible un **volumen de navegación**. Este debe representar la **totalidad del espacio libre** del entorno y no solo las superficies, logrando garantizar que ninguna ruta viable o alternativa óptima quede excluida durante el cálculo de trayectorias.

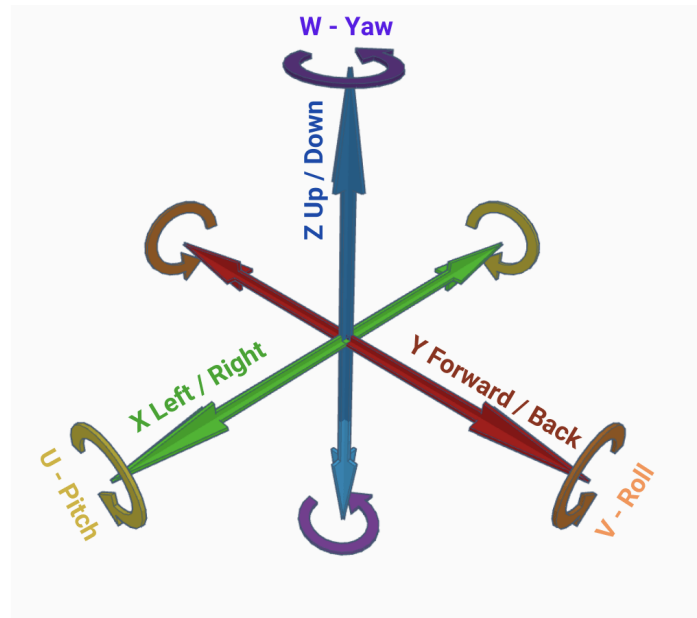


Figura 1.2: Visualización de los 6 grados de libertad. Imagen extraída de [11].

Por consiguiente, el salto hacia una navegación puramente volumétrica plantea un desafío técnico de considerable magnitud. Al añadir la altura como una dimensión libre, la gran cantidad de espacio a analizar dispara el coste computacional tanto en tiempo como en memoria [12]. Si esta complejidad no se gestiona mediante estructuras de datos espaciales altamente especializadas como *octrees* [13] o representaciones jerárquicas equivalentes, el cálculo de trayectorias provocará un consumo crítico de memoria y una degradación severa en la tasa de **Fotogramas Por Segundo (FPS)**, comprometiendo la viabilidad comercial del producto.

Ante la falta de un soporte oficial eficiente en los motores dominantes, los estudios de desarrollo se ven obligados a implementar soluciones *ad hoc* para cada proyecto, o bien a intentar adaptar de forma artificial las tecnologías 2.5D existentes [14]. Ambas alternativas suelen derivar en un incremento significativo de los costes de producción, así como en una

pérdida de tiempo que podría dedicarse al diseño del propio videojuego. Surge, por lo tanto, la necesidad evidente de diseñar un sistema **genérico, optimizado y reutilizable** que resuelva el problema de la navegación tridimensional de manera estructural.

1.2 Objetivos

El objetivo general de este Trabajo de Fin de Grado consiste en el diseño, implementación y validación en el motor Unity de un sistema de navegación volumétrica eficiente y optimizado. El resultado final tendrá un enfoque de producto, materializándose en un paquete de herramientas modular, de código abierto, listo para su integración inmediata en proyectos y entornos de producción reales.

A partir de este objetivo general, se establecen los siguientes objetivos específicos:

- **Diseño arquitectónico:** Diseñar una arquitectura modular y escalable que garantice la eficiencia computacional y facilite la futura integración de nuevas funcionalidades.
- **Representación eficiente del espacio:** Implementar una estructura de datos espacial que represente el volumen navegable de forma compacta y optimizada para consultas en tiempo real.
- **Sistema de serialización:** Desarrollar un módulo de almacenamiento en disco para guardar la representación generada, evitando el coste de su reconstrucción en tiempo de ejecución.
- **Búsqueda de rutas:** Adaptar un algoritmo de búsqueda heurística tridimensional para hallar trayectorias viables dentro del espacio libre del volumen.
- **Postprocesado de trayectorias:** Desarrollar un algoritmo de suavizado de caminos que optimice la trayectoria resultante mediante la eliminación de giros bruscos y nodos redundantes.
- **Integración en Unity:** Desarrollar una herramienta nativa para el editor del motor que simplifique la configuración del sistema y su visualización mediante Gizmos y paneles personalizados.
- **Accesibilidad y usabilidad:** Proveer una interfaz intuitiva que permita la configuración del sistema tanto a perfiles técnicos como a diseñadores de niveles y artistas.
- **Análisis de rendimiento:** Evaluar cuantitativamente la eficiencia global del sistema y de sus componentes críticos bajo diferentes escenarios y cargas de prueba.

- **Distribución modular:** Empaquetar el sistema final bajo los estándares oficiales de Unity para facilitar su integración en otros proyectos y su potencial publicación en la Unity Asset Store.

1.3 Estructura de la memoria

La presente memoria se organiza en los siguientes capítulos:

Capítulo 1: Introducción. El presente capítulo.

Capítulo 2: Fundamentos. Explica los distintos términos y técnicas requeridas para comprender el proyecto.

Capítulo 3: Estado de la cuestión. Examina las soluciones existentes en la literatura y la industria, incluyendo un análisis de [Debilidades, Amenazas, Fortalezas y Oportunidades \(DAFO\)](#).

Capítulo 4: Metodología y planificación. Detalla el modelo de ciclo de vida de software adoptado, así como la planificación temporal y los recursos empleados.

Capítulo 5: Desarrollo. Profundiza en las distintas fases involucradas en la implementación del sistema.

Capítulo 6: Conclusiones. Sintetiza los resultados obtenidos y evalúa las aportaciones del sistema al desarrollo de videojuegos.

Capítulo 7: Trabajos futuros. Propone extensiones y nuevas líneas de investigación que pueden desarrollarse a partir del sistema implementado.

Capítulo 2

Fundamentos

Para abordar el desarrollo de un sistema de navegación volumétrica, resulta imprescindible una comprensión profunda de tanto las tecnologías con las que se va a trabajar como los principios matemáticos o algorítmicos que se usarán en el sistema. En consecuencia, el presente capítulo se divide en dos grandes bloques. En primer lugar, se tratarán los fundamentos tecnológicos, detallando la arquitectura del motor gráfico seleccionado y las herramientas de alto rendimiento necesarias para garantizar su viabilidad y funcionamiento óptimo. En segundo lugar, se profundizará en los fundamentos teóricos, los cuales establecerán las bases de las estructuras de datos y las técnicas algorítmicas empleadas en el proyecto.

2.1 Fundamentos tecnológicos

La creación de un sistema no solo funcional, sino también robusto y eficiente, exige su implementación dentro de un ecosistema de desarrollo maduro que proporcione las herramientas requeridas. En este contexto, ha sido seleccionado el motor de desarrollo de propósito general Unity [15], introducido en el mercado en el 2005, cuyo uso se justificará detalladamente en la Sección 3.2.1 tras un estudio del mercado y alternativas existentes. A continuación se describen sus componentes arquitectónicos y tecnologías más relevantes para el proyecto.

2.1.1 Unity y su modelo de componentes

Unity emplea una arquitectura basada en el patrón *component*, la cual prioriza la composición frente a la herencia jerárquica tradicional de la Programación Orientada a Objetos (POO). En este paradigma, toda entidad presente en el entorno virtual, ya sea un personaje, un elemento decorativo o un elemento de User Interface (UI), se instancia como un objeto genérico de la clase `GameObject`.

Para dotar de comportamiento a estas entidades, se les adjuntan múltiples componentes modulares que heredan de la clase `MonoBehaviour`. Esta modularidad favorece la

creación de sistemas altamente desacoplados y reutilizables. El ciclo de vida de estos componentes se rige por el patrón *game loop*: tras una fase inicial de configuración mediante los métodos `Awake` y `Start`, el motor ejecuta de manera repetida las funciones `Update`, `LateUpdate` y `FixedUpdate`. El programador debe definir en este ciclo continuo la lógica que se ejecutará en cada frame, como la actualización de la trayectoria del agente volador.

Finalmente todos los `GameObject` se agrupan en una escena de manera jerárquica siguiendo una estructura de árbol (véase la Figura 2.1). Esta constituirá el entorno de ejecución del sistema.

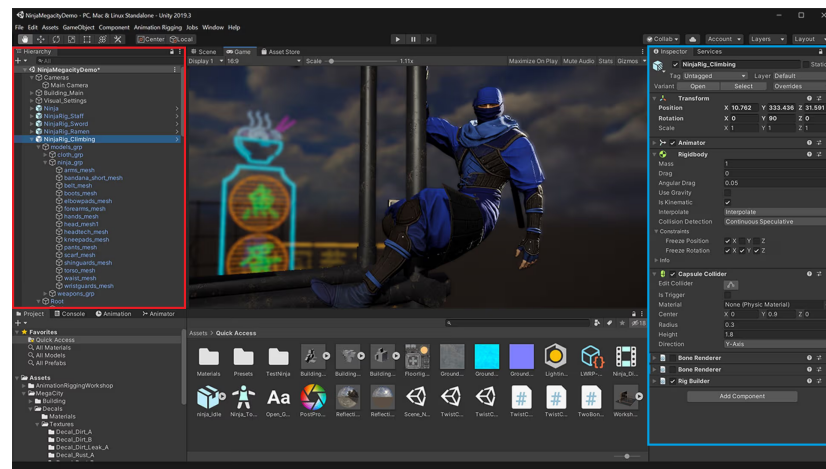


Figura 2.1: Interfaz del editor de Unity. A la izquierda (en rojo) jerarquía de `GameObject` y a la derecha (en azul) inspector de componentes de un objeto. Imagen extraída de [16].

2.1.2 Herramientas proporcionadas por el editor

Un requisito fundamental es facilitar a perfiles no técnicos, como artistas o diseñadores de niveles, la posibilidad de configurar parámetros complejos sin necesidad de alterar el código fuente. Para ello la *Application Programming Interface (API)* de Unity proporciona una serie de herramientas. Mediante el uso de atributos como `[SerializeField]` o `[Range]`, el programador puede exponer en el editor de Unity variables internas de los componentes de modo que estas puedan ser editadas desde fuera. Adicionalmente heredando de la clase `UnityEditor.Editor`, es posible reescribir por completo la interfaz del sistema, añadiendo paneles personalizados, gráficos o validaciones de datos. A estas capacidades se le suma la clase `Gizmos`, una herramienta de depuración visual que permite renderizar primitivas geométricas (por ejemplo, proyectando la malla de colisión), facilitando la validación del sistema sin interferir en el producto final.

2.1.3 Motor de físicas y consultas espaciales

El primer paso para que un agente pueda navegar autónomamente por un entorno tridimensional, es la extracción de información geométrica del escenario. Unity delega la detección de colisiones en su motor de físicas integrado, basado en la biblioteca NVIDIA PhysX [17]. Este sistema utiliza *colliders* (volúmenes matemáticos como cajas, cápsulas o mallas tridimensionales) para representar físicamente a los objetos. Estos pueden ser categorizados en capas con el objetivo de filtrar interacciones entre estas.

La API de Unity proporciona consultas espaciales que permiten consultar la ocupación del entorno. Por ejemplo, la función `OverlapBox` devuelve el conjunto de *colliders* que intersectan un volumen cúbico determinado, utilizando máscaras de bits para restringir esta consulta a las capas de interés. No obstante, en el contexto de la navegación volumétrica, la gran cantidad de consultas requeridas para mapear el espacio aéreo supondría un cuello de botella, agravado por la restricción arquitectónica de Unity de ejecutar la API de físicas de forma síncrona en el hilo principal [18]. Para solventar este impedimento, se hace uso de estructuras como el `OverlapBoxCommand`, la cual permiten agrupar múltiples consultas físicas (*batches*), para su posterior ejecución asíncrona y paralelizada.

2.1.4 Data-Oriented Technology Stack

Históricamente, Unity se ha fundamentado en la POO pura. No obstante, en muchos sistemas que demandan procesamiento masivo de datos, las indirecciones de memoria, dispersión de los objetos en la *heap* y el empaquetado ineficiente de datos provocan fallos continuos de caché degradando severamente el rendimiento.

Para combatir esta limitación, Unity ha desarrollado el ecosistema *Data Oriented Technology Stack* (DOTS) [19]. Este paradigma, basado en el *Diseño Orientado a Datos* (DOD), busca maximizar el rendimiento estructurando los datos en memoria de forma que la *Central Processing Unit* (CPU) pueda procesar la información de manera óptima aprovechando la arquitectura del hardware (ilustrado en la Figura 2.2).

Los componentes clave de este *stack* tecnológico son los siguientes:

- **C# Job System:** API que abstrae y simplifica la programación concurrente, permitiendo y simplificando la ejecución segura de código multihilo. Funciona mediante unidades de cómputo que reciben el nombre de *jobs*, las cuales declaran explícitamente sus dependencias de datos [21]. Esta arquitectura previene errores en tiempo de ejecución como es el caso de las *race conditions*.
- **Burst Compiler:** compilador avanzado diseñado para traducir el lenguaje C# directamente a código máquina nativo, eludiendo así la máquina virtual estándar de este len-

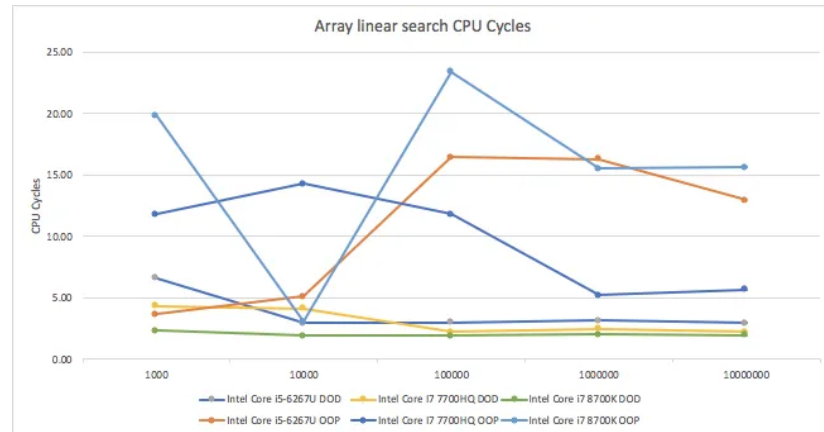


Figura 2.2: Comparativa de POO y DOD. Imagen extraída de [20].

guaje conocida como **Common Language Runtime (CLR)** [22]. Basado en la infraestructura **LLVM** [23], optimiza el código aprovechando las instrucciones **Single Instruction, Multiple Data (SIMD)** que permiten que el procesador ejecute una misma operación matemática simultáneamente sobre un conjunto de datos, acelerando drásticamente los cálculos.

- **Unity.Mathematics:** biblioteca matemática diseñada para aprovechar al máximo el Burst Compiler [24]. Proporciona una serie de tipos de datos vectoriales como el *float3* o *int3*.
- **Colecciones nativas:** Estructuras de datos (tales como el `NativeArray`, `NativeList` o `NativeParallelMultiHashMap`) que reemplazan las colecciones estándar de C#. Estas operan sobre memoria no administrada lo que permite suprimir picos de latencia provocados por el recolector de basura. Al usar estas colecciones, el programador asume el control absoluto y la responsabilidad de reservar y liberar explícitamente la memoria [25].

2.1.5 Características de bajo nivel en C#

Para complementar el ecosistema de **DOTS** y asegurar un rendimiento óptimo, resulta indispensable emplear características de bajo nivel del lenguaje. Las siguientes técnicas minimizan el impacto del recolector de memoria y reducen costes de transferencia de datos en zonas de ejecución críticas:

- **ref readonly:** permite devolver una referencia de solo lectura a una estructura de datos. Al evitar la copia de estructuras complejas, se reduce notablemente el uso de me-

moria. Adicionalmente, la palabra clave `readonly` garantiza a nivel de compilación la integridad de los datos originales.

- **Span<T>**: representa una sección de memoria contigua independientemente de si los datos residen en el `stack` o en la `heap`. Esta estructura permite realizar operaciones de lectura y escritura sobre secciones de esta sin necesidad de almacenar más memoria.
- **stackalloc**: permite reservar memoria en el `stack`. De este modo evita al recolector de basura tener que limpiar colecciones que se usan a modo de *buffers* temporales, asegurando su liberación inmediata al salir de la función y omitiendo por completo al recolector de basura.

A este conjunto de técnicas se añade la manipulación de bits. Esta técnica permite codificar varios campos en una única variable entera, reduciendo drásticamente el uso de memoria.

TODO hacer referencia a cuando estos se comenten en la fase de desarrollo a modo de ejemplo.

2.2 Fundamentos teóricos

Para el diseño de una herramienta de navegación tridimensional robusta, la elección de las tecnologías de bajo nivel constituye únicamente la mitad del problema. La otra mitad reside en la selección e implementación de los algoritmos y estructuras de datos adecuados que dictaminarán la eficiencia y rendimiento del sistema. A continuación, se describen los principios teóricos que constituyen el campo de la navegación volumétrica.

2.2.1 Búsqueda de caminos

La búsqueda de caminos, o *pathfinding*, consiste en el problema computacional de determinar una ruta continua y libre de colisiones que conecte un punto de origen con un destino dentro de un entorno virtual [3]. Dado que el espacio físico es continuo y contiene infinitas posiciones posibles, el primer paso para su resolución exige obtener una representación discreta y estructurada de la geometría de la escena. Esta abstracción permitirá aplicar, posteriormente, un algoritmo de búsqueda de caminos mínimos.

Representación del espacio navegable

Al estudiar el ámbito de la navegación volumétrica, se puede observar que existen tres alternativas predominantes. Estas se categorizan fundamentalmente en:

- **Waypoints:** Esta solución requiere el posicionamiento manual de una serie de nodos que reciben dicho nombre. Estos puntos constituirán los vértices del grafo y deben ubicarse estrictamente dentro del espacio libre de colisiones. Las aristas se corresponderán a aquellas conexiones de vértices para las cuales existe una línea de visión directa, es decir, sin obstáculos intermedios entre ellos.

A pesar de ser computacionalmente el método más ligero, este carece de escalabilidad en escenarios de gran tamaño o en entornos dinámicos, ya que cualquier cambio en la geometría de la escena obliga a recolocar los nodos y recalcular sus conexiones. Sin embargo, dado que la implementación de un sistema con estas características es trivial, es común encontrar estudios que recurren a soluciones *ad hoc* cuando no se requieren sistemas de navegación complejos [3].

- **Campos de flujo (*Flow fields*):** Esta aproximación consiste en calcular un campo vectorial global sobre una cuadrícula o volumen espacial, donde cada celda almacena un vector de dirección que apunta hacia el destino. Si bien resultan extremadamente eficientes para movilizar enjambres o multitudes masivas hacia un objetivo común, su coste computacional y uso de memoria en entornos tridimensionales se vuelve prohibitivo cuando existen múltiples agentes persiguiendo destinos independientes, descartándolos para soluciones de propósito general [26].
- **Octrees:** Constituyen la técnica más extendida en la industria. En ella, el espacio se subdivide en octantes, creando una estructura de datos jerárquica que representa el espacio libre navegable [13]. En prácticamente la totalidad de los casos, estas representaciones se optimizan haciendo uso de una modificación estructural: los *Sparse Voxel Octree (SVO)*. Estos dividen solo aquellos octantes que contienen geometría, logrando así un ahorro de memoria considerable.

La discretización del volumen mediante estas estructuras requiere localizar rápidamente las celdas o *vóxels* en la memoria. Para lograrlo, resulta fundamental mapear coordenadas tridimensionales (X, Y, Z) a un índice escalar, preservando la localidad espacial. Esto se consigue mediante el cálculo de códigos de Morton, basados en las curvas de orden Z [27] (véase la Figura 2.3). A través de operaciones a nivel de bits, se intercalan los bits representativos de cada coordenada espacial para formar un único número entero. Esta técnica garantiza que nodos espacialmente adyacentes en la escena tiendan a ocupar posiciones contiguas en la memoria lineal, reduciendo drásticamente los fallos de caché y maximizando los beneficios arquitectónicos del paradigma *DOD* ya explicados en la Sección 2.1.4.

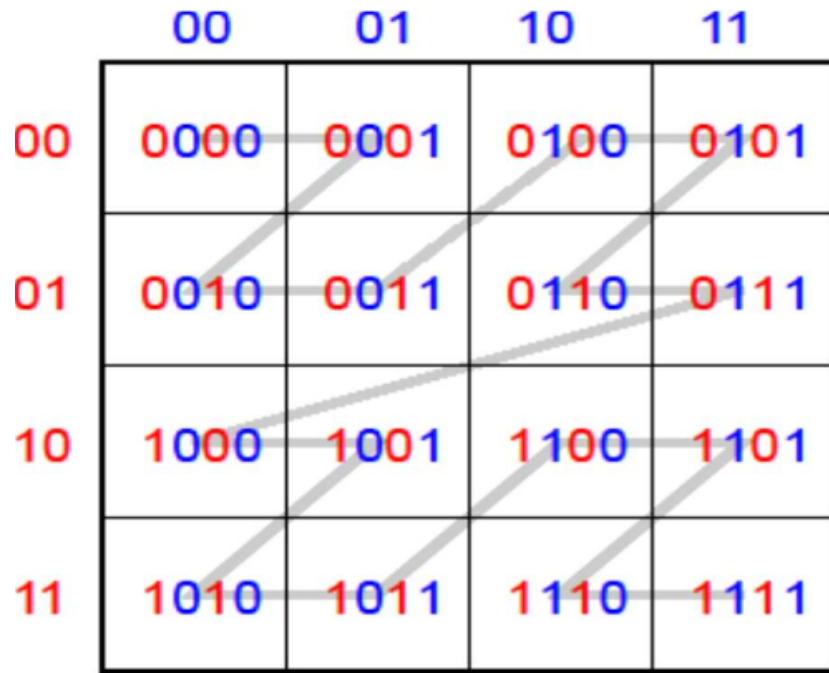


Figura 2.3: Versión 2D de los códigos de Morton. Imagen extraída de [28].

Algoritmos de búsqueda en grafos

Una vez estructurado el espacio en un grafo navegable, la extracción de la trayectoria se delega en algoritmos de caminos mínimos. El algoritmo clásico de Dijkstra explora los nodos de forma uniforme garantizando siempre la ruta más corta [29]. No obstante, su expansión radial conlleva un coste computacional inasumible para cálculos en tiempo real dentro de grandes volúmenes.

En respuesta a esta limitación, el estándar de la industria es el algoritmo A^* . Esta técnica optimiza la búsqueda introduciendo una heurística a la función de coste. Dicha función se define matemáticamente como $f(n) = g(n) + h(n)$, donde $g(n)$ es el coste real desde el origen y $h(n)$ es el coste estimado hasta la meta. Esta estimación permite priorizar los nodos más prometedores, orientando la búsqueda hacia el destino y reduciendo exponencialmente el número de celdas evaluadas [9].

Postprocesado de trayectorias

Las rutas generadas por algoritmos sobre volúmenes discretizados resultan muy artificiales al estar compuestas por multitud de segmentos. Para dotar al agente de un movimiento orgánico, el sistema requiere de una etapa de postprocesado que se suele dividir en dos fases.

En primer lugar, se aplica un algoritmo de *String Pulling* con su variante voraz. Este itera sobre la ruta buscando el nodo más lejano con el que exista línea de visión directa, descartando

recursivamente todos los nodos intermedios redundantes. Para validar esta línea de visión se requiere trazar rayos y comprobar su intersección con la geometría. Esto requiere que el algoritmo no sea solo correcto sino que también responda de manera robusta a errores derivados de la falta de precisión decimal, siendo el algoritmo [Digital Differential Analyzer \(DDA\)](#) [30] el caso estándar de aplicación.

En segundo lugar, la trayectoria depurada se suaviza mediante la interpolación con curvas paramétricas. Se suele optar por el *spline* de Catmull-Rom (como se muestra en la Figura 2.4) sobre alternativas como las curvas de Bézier o B-splines. Esta decisión se fundamenta en que este tipo de curvas pasa de forma exacta a través de todos los puntos de control suministrados [31].

Como se ilustra en la figura, los puntos rojos (denotados con la letra P) representan estos **puntos de control** o nodos de la trayectoria original. Dado que la etapa previa asegura que estos nodos están ubicados en espacio libre, obligar a la curva a cruzar exactamente por ellos, garantizando que la trayectoria resultante no interseque accidentalmente con la geometría del entorno.

Por su parte, la curva negra y las etiquetas Q representan los **segmentos** generados matemáticamente que unen dichos puntos de control. El trazado de estos segmentos se define mediante el parámetro de interpolación u (ilustrado con marcas amarillas), el cual toma valores entre 0 y 1 para calcular las posiciones intermedias a lo largo del segmento, permitiendo así una transición suave entre un punto de control y el siguiente.

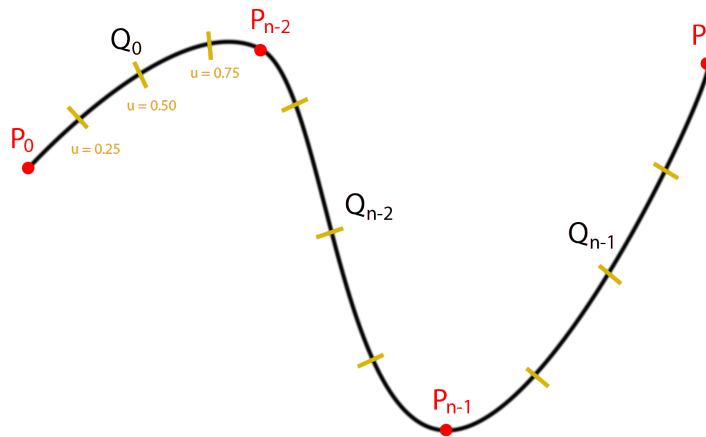


Figura 2.4: Ejemplo curva Catmull Rom interpolando los puntos de control. Imagen extraída de [32].

2.2.2 Planificación de caminos

A pesar de haber encontrado un camino libre de obstáculos, en múltiples ocasiones se requiere la replanificación de este, por ejemplo, para esquivar a otros agentes. A este proceso se le conoce como *path planning*, siendo el algoritmo [Optimal Reciprocal Collision Avoidance \(ORCA\)](#) uno de los más relevantes [33]. Este algoritmo se basa en los llamados obstáculos de velocidad, correspondiéndose estos con los propios agentes dinámicos de la escena. Para lograr la evasión, [ORCA](#) razona sobre el denominado *espacio de velocidades*: un dominio continuo en el que cada punto representa un posible vector de velocidad que el agente podría adoptar en el instante actual. Dentro de este dominio, el algoritmo delimita la región de velocidades **prohibidas** (aquellas que, de mantenerse, conducirían a una colisión) frente a las velocidades **permitidas**, consideradas seguras. La particularidad del método reside en su premisa de reciprocidad: ambos agentes ejecutan [ORCA](#) de forma simultánea y asumen de manera conjunta la responsabilidad de esquivarse, en lugar de delegar toda la maniobra en uno solo de ellos.

A modo de ejemplo ilustrativo, considérense dos agentes voladores, A y B , que se aproximan frontalmente a lo largo de una misma recta. La velocidad preferida de A , aquella que lo dirige directamente hacia su destino, apunta hacia B , por lo que recae dentro de su región de velocidades prohibidas: mantenerla garantizaría el impacto. El algoritmo calcula entonces el mínimo vector de corrección $\vec{u}$ que situaría la velocidad relativa de ambos justo sobre la frontera de la región segura, por ejemplo, desviándose ligeramente hacia un lado. Si B fuese un obstáculo estático incapaz de reaccionar, A tendría que asumir esa corrección en su totalidad. No obstante, dado que B también ejecuta el algoritmo, la responsabilidad se reparte de forma equitativa: A modifica su velocidad en $\vec{u}/2$ y B aplica la otra mitad en sentido opuesto. Así, ambos describen trayectorias simétricas y la colisión se evita con la mitad del esfuerzo por cada parte, lo que elimina además las oscilaciones que surgirían si cada agente supusiese que el otro no va a alterar su rumbo.

En términos operacionales, [ORCA](#) modela matemáticamente estas restricciones definiendo un semiplano de velocidades seguras para cada par de agentes en conflicto. A partir de la posición relativa, los radios de colisión y las velocidades actuales, se computa la mínima alteración de velocidad necesaria para garantizar que las trayectorias no se intersequen. Gracias a la premisa de reciprocidad, el algoritmo asume que cada entidad aplicará exactamente la mitad de esta corrección. Para ello, el sistema emplea técnicas de programación lineal para calcular de manera eficiente una nueva velocidad óptima que satisfaga todas las restricciones geométricas de los semiplanos y, simultáneamente, se aproxime lo máximo posible a la velocidad preferida del agente (aquella que lo dirige hacia su meta).

El principal problema de este sistema de evasión local es que los agentes se pueden quedar atascados ante obstáculos dinámicos muy grandes. Resolver este problema, denominado

`deadlock`, conlleva hacer uso de sistemas de planificación que trabajen a más alto nivel y que operen independientemente del sistema de navegación, lo cual escapa del alcance de este proyecto.

2.2.3 Integridad estructural mediante algoritmos Hash

Con el propósito de evitar reconstruir la representación del espacio navegable cada vez que se cargue el juego, los sistemas de navegación suelen proveer un proceso denominado como `bake`. Este consiste en construir las estructuras de datos de manera anticipada desde el editor y guardarlas en disco para que, posteriormente, puedan ser cargadas en tiempo de ejecución.

Para asegurar la integridad de la escena, es decir, garantizar que no se han producido cambios en su geometría desde la última vez que se realizó el `bake`, se suele computar el *hash* de los objetos correspondientes. De este modo, si el valor resultante varía, se detecta que se ha producido una modificación. En el ecosistema de C#, no se recomienda utilizar el método por defecto `GetHashCode` [34]. Esto se debe a que proporciona resultados dependientes de la semilla de ejecución, por lo que los valores pueden variar entre diferentes sesiones, rompiendo por completo el determinismo.

En su lugar, se suelen implementar algoritmos de *hash* desde cero, como es el caso de `FNV-1a`, el cual destaca por ser extremadamente rápido [35]. Este algoritmo itera directamente sobre los bytes de los datos, acumulando el resultado en un valor inicializado con el llamado *FNV offset basis*. En cada iteración, aplica una operación XOR con el byte correspondiente y, acto seguido, lo multiplica por un número primo conocido como *FNV prime*.

Estado de la cuestión

El presente capítulo ofrece una revisión detallada del panorama actual respecto a las soluciones de navegación espacial existentes en el ámbito de los videojuegos. A lo largo de las siguientes secciones se explorarán las herramientas con mayor relevancia en el mercado actual. Posteriormente, a partir de un análisis crítico de estas, se justificará la necesidad de la herramienta de código abierto propuesta en este trabajo.

3.1 Herramientas disponibles

En la actualidad, existe un amplio repertorio de herramientas orientadas a resolver el problema de la navegación en entornos virtuales. Entre estas soluciones, aquellas que gozan de mayor relevancia y adopción en la industria, se fundamentan en enfoques que combinan estructuras *SVO* y el algoritmo de búsqueda heurística *A** [3], conceptos ya explicados previamente en la Sección 2.2.1. A nivel arquitectónico, estas soluciones se dividen en dos grandes grupos: aquellas que operan de forma independiente al motor gráfico y las diseñadas para integrarse en un entorno de desarrollo concreto.

3.1.1 Herramientas genéricas

Si se analizan el conjunto de herramientas que son agnósticas al motor, se observa que la solución predominante en el mercado es la de Mercuna [36]. Esta herramienta fue concebida originalmente como un *plugin* comercial exclusivo para Unreal Engine [37]. Sin embargo, su éxito motivó su posterior extensión para operar de forma desacoplada a este.

El principal atractivo de Mercuna radica en su capacidad para unificar, bajo un mismo ecosistema, prácticamente la totalidad de los casos de uso que engloba la navegación. Entre sus características más relevantes se encuentran:

- **Navegación volumétrica:** Permite a los agentes tanto voladores como submarinos, moverse por la totalidad del espacio navegable, haciendo uso completo de sus seis grados de libertad.
- **Navegación sobre superficies:** Proporciona la capacidad de desplazarse sobre cualquier tipo de geometría. Esto no se limita al suelo convencional, sino que incluye también paredes o techos, gestionando la gravedad de manera local al agente (ver Figura 3.1).
- **Soporte para agentes no holonómicos:** Incorpora las restricciones físicas y cinemáticas de los agentes en la fase de cálculo de trayectorias. De este modo, es posible modelar el comportamiento de vehículos con radios de giro limitados o entidades que carecen de la capacidad de acelerar y frenar de manera instantánea.

Se trata, sin lugar a dudas, de la herramienta que se puede considerar más robusta y completa del mercado. Sin embargo, su modelo de negocio supone una barrera de entrada prohibitiva para la mayor parte de estudios. Mercuna dispone de distintas licencias que dependerán del subconjunto de características requerido, con costes que parten de £15000 o de £2500 para estudios con un presupuesto de desarrollo inferior a \$500000 [38].

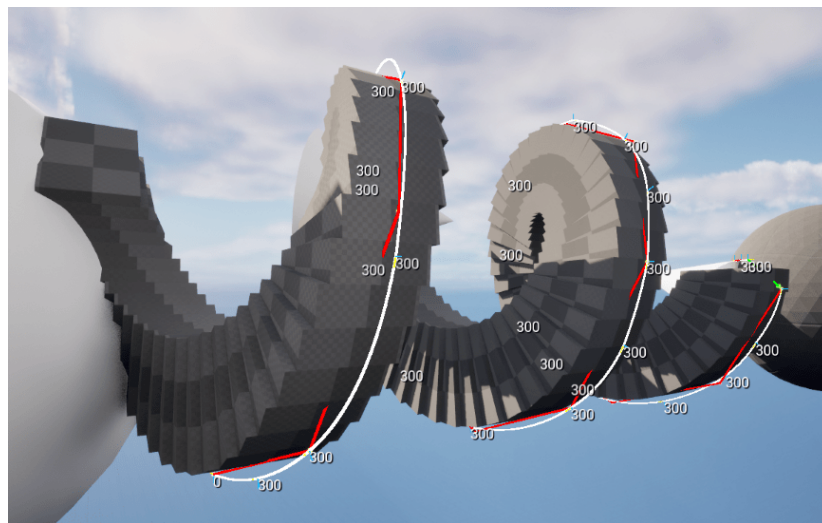


Figura 3.1: Visualización de navegación por cualquier superficie. Imagen extraída de [39].

3.1.2 Soluciones específicas de motor

Por el contrario, las herramientas especializadas en un motor gráfico concreto, operan en un paradigma de desarrollo diferente. Al prescindir de una arquitectura genérica y agnóstica que asegure su interoperabilidad multiplataforma, estas soluciones pueden aprovechar las

rutinas nativas del motor para optimizar el rendimiento computacional, ofreciendo asimismo una integración total con el editor. Esto se traduce en herramientas significativamente más intuitivas para el desarrollador final.

En este ámbito predomina el ecosistema de Unity, fuertemente respaldado por su tienda oficial de recursos: la Asset Store [40]. Esta plataforma permite comercializar tanto herramientas de desarrollo como recursos artísticos (modelos tridimensionales, texturas o composiciones musicales, entre otros) que aceleran los flujos de trabajo al evitar la necesidad de diseñar dichos componentes desde el principio. Bajo un modelo de reparto de ingresos, Unity proporciona visibilidad de los productos creados.

Como consecuencia de la vasta magnitud de este ecosistema, prácticamente la totalidad de las herramientas de terceros se desarrollan de manera exclusiva para este motor. Mientras que en entornos como Unreal Engine las soluciones complejas suelen comercializarse con costosas licencias comerciales, la Asset Store de Unity proporciona productos más asequibles para estudios independientes o de menor presupuesto. Entre estas destacan:

Nav3D (Softbrix Studio)

Desarrollada por Softbrix Studio, Nav3D es la opción que ostenta mejores valoraciones en la tienda de Unity (Asset Store) [41]. Su mayor virtud reside en que no se limita al cálculo de rutas estáticas, sino que integra un sistema de evasión local. Esto garantiza que los NPC, en escenarios multiagente, coordinen sus trayectorias y las reajusten en tiempo real para evitar colisionar entre ellos [42].

Nav3D (m6io)

Creada por el desarrollador independiente m6io, esta solución se comercializa con un enfoque puramente técnico. A pesar de cubrir la mayoría de casos de uso, carece por completo de herramientas visuales e interfaces de configuración del editor. En consecuencia, presenta una curva de aprendizaje notablemente más pronunciada, exigiendo al usuario un nivel avanzado de programación en el lenguaje C# [43].

HyperNav

Elaborada por Infohazard Games, constituye actualmente una de las alternativas más robustas y sofisticadas del mercado de Unity, lo cual se refleja también en un coste superior. Su principal ventaja competitiva es su enfoque multiparadigma: además de la navegación puramente volumétrica, ofrece navegación sobre superficies, permitiendo a los agentes desplazarse por suelos y paredes con una gestión automatizada de la gravedad local [44].

Spatial Pathfinding

Desarrollada por TecMaid, se presenta como una alternativa más económica para desarrolladores con presupuestos ajustados. Se trata de una solución que sacrifica funcionalidades avanzadas en favor de ofrecer un sistema más ligero, directo y de implementación rápida [45].

3.2 Comparativa de mercado

A modo de síntesis, la herramienta de carácter agnóstico Mercuna ofrece una solución tecnológicamente superior, si bien su modelo comercial la orienta exclusivamente a producciones de alto presupuesto conocidas como AAA. Por otro lado, las alternativas nativas para Unity democratizan el acceso a estas tecnologías con precios mucho más asequibles, situándose por lo general por debajo de los 100 euros.

Sin embargo, estas últimas opciones se distribuyen bajo modelos de **código cerrado**, lo que supone una desventaja técnica importante. Esta opacidad impide a los desarrolladores consultar el código y prever el impacto real de cada funcionalidad en el rendimiento, lo que habitualmente deriva en un uso subóptimo del sistema. Además, en muchos casos, estas soluciones carecen de un nivel de refinamiento adecuado en cuanto a usabilidad; como es el caso del ya mencionado anteriormente Nav3D de m6io, que prescinde por completo de herramientas gráficas. Esta comparativa general queda reflejada de manera resumida en la Tabla 3.1.

Herramienta	Plataforma	Enfoque	Precio
Mercuna	Multiplataforma	Multiparadigma	£15000 +
Nav3D (Softbrix Studio)	Unity	Volumétrica	64,39€
Nav3D (m6io)	Unity	Volumétrica	59,80€
HyperNav	Unity	Volumétrica + Superficies	82,80€
Spatial Pathfinding	Unity	Volumétrica	27,59€

Tabla 3.1: Resumen de las alternativas existentes

3.2.1 Justificación del entorno de desarrollo

En base a las soluciones analizadas, la elección de Unity como entorno para de desarrollo resulta óptima para este trabajo. El ecosistema de la Asset Store garantiza no solo que la solución responda a una demanda real, sino que también asegura un canal con la visibilidad suficiente para su correcta difusión.

A este factor se suma una ventaja técnica fundamental: la disponibilidad de herramientas orientadas al alto rendimiento integradas de forma nativa en el motor, como es el caso del Burst Compiler. Esta tecnología permite compilar código C# en código máquina altamente optimizado mediante el uso de la infraestructura LLVM [23]. Dado que la navegación volumétrica implica una carga computacional intensiva, aprovechar el Burst Compiler facilita la ejecución de estas rutinas matemáticas en *hilos secundarios* con un impacto mínimo en el *hilo principal de procesamiento*, preservando así la tasa de refresco del entorno interactivo.

3.3 DAFO

Con el propósito de evaluar la viabilidad estratégica y justificar el desarrollo del proyecto, se ha llevado a cabo un análisis DAFO. Este estudio se estructura en un bloque de **análisis interno** (orientado a identificar las fortalezas y debilidades inherentes a la propuesta), un bloque de **análisis externo** (centrado en discernir las oportunidades y amenazas que presenta el mercado actual) y un apartado de conclusiones.

3.3.1 Análisis interno

La principal fortaleza del sistema propuesto radica en su condición de ser la primera herramienta de código abierto específicamente orientada a la navegación volumétrica. A diferencia de las alternativas comerciales cerradas, este enfoque permite a los usuarios auditar el sistema, adaptarlo mediante modificaciones personalizadas según las necesidades de su proyecto y, potencialmente, contribuir a la evolución de este. Asimismo, la transparencia en el diseño del algoritmo permite a los desarrolladores prever con precisión el impacto en el rendimiento de cada componente.

Por el contrario, la principal debilidad se asocia a su naturaleza como proyecto emergente dentro de un mercado maduro y consolidado por soluciones veteranas. Esto puede generar una percepción inicial de falta de madurez o robustez, lo que exigirá un proceso continuo de iteración y refinamiento técnico basado en la retroalimentación recibida tras las fases iniciales de despliegue.

3.3.2 Análisis externo

Las oportunidades del proyecto emergen ante la carencia de soluciones integradas o gratuitas de navegación volumétrica en las versiones nativas de los motores gráficos actuales. En este contexto, la existencia de un mercado centralizado como la Asset Store, permitirá dar a conocer la herramienta directamente a desarrolladores independientes.

Por último, las amenazas provienen del dinamismo y la constante evolución tecnológica que caracterizan al sector de los videojuegos. El auge de motores alternativos de código

abierto, como Godot [46], los cuales incorporan continuamente nuevas capacidades [47], representa un factor de competitividad externa. Asimismo, la eventual inclusión de una herramienta de navegación tridimensional oficial y nativa por parte de la propia compañía Unity constituye el riesgo más significativo para soluciones desarrolladas por terceros.

Fortalezas	Debilidades
• Primera herramienta de código abierto en su campo. • Transparencia total, permitiendo optimización y adaptabilidad por parte del usuario.	• Proyecto emergente en un mercado con alternativas consolidadas. • Necesidad inicial de cambios para adaptarse a los requisitos de la comunidad.
Oportunidades	Amenazas
• Carencia de herramientas integradas gratuitas de navegación volumétrica. • Gran canal de difusión directa a través de la Asset Store.	• Sector de los videojuegos en constante cambio y evolución. • Crecimiento de motores emergentes competitivos (ej. Godot). • Riesgo de aparición de una herramienta oficial integrada en el motor.

Tabla 3.2: Matriz DAFO del proyecto

3.3.3 Conclusiones

A partir de la matriz DAFO expuesta en la Tabla 3.2, se concluye que el desarrollo de este sistema es altamente viable. Si bien el proyecto deberá competir con soluciones comerciales maduras, su naturaleza de código abierto representa una ventaja competitiva crítica. En definitiva, el proyecto se posiciona como una alternativa democratizadora y accesible, con un claro potencial de adopción dentro del vasto ecosistema de desarrolladores de Unity.

Lista de acrónimos

API Application Programming Interface. [7](#), [8](#)

CLR Common Language Runtime. [9](#)

CPU Central Processing Unit. [8](#)

DAFO Debilidades, Amenazas, Fortalezas y Oportunidades. [i](#), [5](#), [20](#), [21](#)

DDA Digital Differential Analyzer. [13](#)

DOD Diseño Orientado a Datos. [iii](#), [8](#), [9](#), [11](#)

DOTS Data Oriented Technology Stack. [8](#), [9](#)

FPS Fotogramas Por Segundo. [3](#)

NPC Non-Playable Character. [1](#), [18](#)

ORCA Optimal Reciprocal Collision Avoidance. [14](#)

POO Programación Orientada a Objetos. [iii](#), [6](#), [8](#), [9](#)

SIMD Single Instruction, Multiple Data. [9](#)

SVO Sparse Voxel Octree. [11](#), [16](#)

UI User Interface. [6](#)

Glosario

A* Algoritmo de búsqueda de caminos en grafo que hace uso de heurísticas. [2](#), [12](#), [16](#)

AAA Clasificación para videojuegos desarrollados con un alto presupuesto. [19](#)

ad hoc Expresión para referirse a que se hace con un fin determinado. [3](#), [11](#)

bake Proceso de precomputación realizado durante la fase de desarrollo (en el editor). Consiste en calcular de forma anticipada información costosa —como la generación de mallas de navegación, estructuras espaciales o iluminación estática— y almacenarla en disco, evitando así realizar dichos cálculos durante el tiempo de ejecución del sistema. [15](#)

deadlock Situación de interbloqueo temporal o permanente. En el contexto de la navegación de agentes, ocurre cuando dos o más entidades bloquean mutuamente sus respectivas trayectorias, impidiendo que cualquiera de ellas pueda avanzar hacia su destino al verse incapaces de resolver la evasión geométrica de forma local. [15](#)

FNV-1a Función hash no criptográfica diseñada por Fowler, Noll y Vo, conocida por su rapidez y alta tasa de dispersión. Se utiliza frecuentemente en tablas hash y aplicaciones donde el rendimiento es crítico, ya que procesa los datos de forma eficiente mediante operaciones simples a nivel de bits (XOR y multiplicación). [15](#)

heap Región de memoria utilizada para la asignación dinámica. Los bloques de memoria se reservan y liberan en tiempo de ejecución, ya sea manualmente o mediante un recolector de basura, sin un orden establecido. [8](#), [10](#)

LLVM Infraestructura de compilación modular diseñada para optimizar el tiempo de compilación, enlace y ejecución de programas. Su nombre no es un acrónimo. [9](#), [20](#)

no holonómico Entidad cuyo movimiento está sujeto a restricciones cinemáticas y físicas, lo que le impide cambiar de dirección o velocidad de manera instantánea (por ejemplo, un vehículo con un radio de giro limitado). [17](#)

octree Estructura de datos en árbol donde cada nodo interno tiene exactamente ocho hijos. Se utiliza principalmente para la partición jerárquica y eficiente del espacio tridimensional.

3, 11

race condition Anomalía que ocurre en la programación concurrente o multihilo cuando dos o más subprocesos intentan acceder y modificar un recurso compartido de forma simultánea. El estado final del sistema se vuelve impredecible, ya que depende del orden exacto e incontrolable en el que el procesador planifique la ejecución de las instrucciones. 8

stack Región de memoria gestionada de forma automática y secuencial (último en entrar, primero en salir). Se utiliza para almacenar variables locales y el contexto de las llamadas a funciones durante la ejecución de un programa. 10

vóxel Acrónimo de *volumetric pixel* (píxel volumétrico). Constituye la unidad cúbica mínima procesable dentro de una matriz o cuadrícula tridimensional. Es el equivalente tridimensional del píxel y se utiliza ampliamente para discretizar y representar la ocupación de espacios volumétricos. 11

Bibliografía

- [1] S. Russell, P. Norvig, and A. Intelligence, “A modern approach,” *Artificial Intelligence. Prentice-Hall, Egnlewood Cliffs*, vol. 25, no. 27, pp. 79–80, 1995.
- [2] C. W. Reynolds *et al.*, “Steering behaviors for autonomous characters,” in *Game developers conference*, vol. 1999. San Francisco, CA, 1999, pp. 763–782.
- [3] I. Millington and J. D. Funge, *Artificial intelligence for games*, 2nd ed. Burlington (Massachusetts): Morgan Kaufmann/Elsevier, 2009.
- [4] Z. Abd Algfoor, M. S. Sunar, and H. Kolivand, “A comprehensive study on pathfinding techniques for robotics and video games,” *International Journal of Computer Games Technology*, vol. 2015, no. 1, p. 736138, 2015.
- [5] Epic Games, “Navigation system in unreal engine,” <https://docs.unrealengine.com/5.3/en-US/navigation-system-in-unreal-engine/>, consultado el 15 de junio de 2026.
- [6] Unity Technologies, “AI navigation package documentation,” <https://docs.unity3d.com/Packages/com.unity.ai.navigation@2.0/manual/index.html>, consultado el 15 de junio de 2026.
- [7] Godot Engine, “Navigationserver3d — godot engine documentation,” https://docs.godotengine.org/en/stable/classes/class_navigationserver3d.html, consultado el 15 de junio de 2026.
- [8] M. Mononen, “Recast navigation,” <https://recastnav.com/>, 2009, consultado el 15 de junio de 2026.
- [9] P. E. Hart, N. J. Nilsson, and B. Raphael, “A formal basis for the heuristic determination of minimum cost paths,” *IEEE Transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968.

- [10] Unity Technologies, “Creación de un agente de NavMesh — manual de unity,” <https://docs.unity3d.com/es/530/Manual/nav-CreateNavMeshAgent.html>, consultado el 15 de junio de 2026.
- [11] Industrial Inspection & Analysis, “What are the 6 degrees of freedom? (6DoF explained),” <https://industrial-ia.com/what-are-the-6-degrees-of-freedom-6dof-explained/>, consultado el 15 de junio de 2026.
- [12] S. M. LaValle, *Planning algorithms*. Cambridge university press, 2006.
- [13] D. Meagher, “Geometric modeling using octree encoding,” *Computer graphics and image processing*, vol. 19, no. 2, pp. 129–147, 1982.
- [14] D. Brewer, “Getting off the NavMesh: Navigating in fully 3D environments,” <https://www.gdcvault.com/play/1022016/Getting-off-the-NavMesh-Navigating>, 2015, game Developers Conference (GDC). consultado el 15 de junio de 2026.
- [15] Unity Technologies, “Unity,” <https://unity.com/>, 2026, consultado el 15 de junio de 2026.
- [16] —, “Editor tools - unity 2019.3,” 2020, accedido: 15 de junio de 2026. [Online]. Available: <https://unity.com/es/releases/2019-3/editor-tools>
- [17] NVIDIA Corporation, “Nvidia physx system software,” 2019, accedido: 15 de junio de 2026. [Online]. Available: <https://www.nvidia.com/es-es/drivers/physx/physx-9-19-0218-driver/>
- [18] Unity Technologies, “Unity physics api documentation,” 2022, accedido: 15 de junio de 2026. [Online]. Available: <https://docs.unity3d.com/Packages/com.unity.physics@6.5/api/index.html>
- [19] —, “Data-oriented technology stack (dots),” 2023, accedido: 15 de junio de 2026. [Online]. Available: <https://unity.com/dots>
- [20] D. Albano, “Data processing in c#: Performance of oop vs dod,” 2019, accedido: 15 de junio de 2026. [Online]. Available: <https://medium.com/@d.albano/data-processing-in-c-performance-of-oop-vs-dod-a23e94babf5c>
- [21] Unity Technologies, “C# job system,” 2023, accedido: 15 de junio de 2026. [Online]. Available: <https://docs.unity3d.com/Manual/JobSystem.html>
- [22] —, “Burst user guide,” <https://docs.unity3d.com/Packages/com.unity.burst@1.8/manual/index.html>, 2026, consultado el 15 de junio de 2026.

- [23] C. Lattner and V. Adve, “LLVM: A compilation framework for lifelong program analysis & transformation,” in *Proceedings of the International Symposium on Code Generation and Optimization (CGO)*. IEEE Computer Society, 2004, pp. 75–86.
- [24] Unity Technologies, “Unity.mathematics package,” 2023, accedido: 15 de junio de 2026. [Online]. Available: <https://docs.unity3d.com/Packages/com.unity.mathematics@1.2/manual/index.html>
- [25] —, “Native collections in unity,” 2023, accedido: 15 de junio de 2026. [Online]. Available: <https://docs.unity3d.com/Manual/JobSystemNativeContainer.html>
- [26] E. Emerson, “Crowd pathfinding and steering using flow field tiles,” in *Game AI Pro 360: Guide to Movement and Pathfinding*. CRC Press, 2019, pp. 67–76.
- [27] G. M. Morton, *A computer oriented geodetic data base and a new technique in file sequencing*. International Business Machines Company, 1966.
- [28] M. Vinkler, J. Bittner, and V. Havran, “Extended morton codes for high performance bounding volume hierarchy construction,” in *Proceedings of High Performance Graphics*, ser. HPG ’17. Association for Computing Machinery, 2017. [Online]. Available: https://www.highperformancegraphics.org/wp-content/uploads/2017/Papers-Session3/HPG207_ExtendedMortonCodes.pdf
- [29] E. W. Dijkstra, “A note on two problems in connexion with graphs,” in *Edsger Wybe Dijkstra: his life, work, and legacy*, 2022, pp. 287–290.
- [30] J. Amanatides, A. Woo *et al.*, “A fast voxel traversal algorithm for ray tracing,” in *Eurographics*, vol. 87, no. 3, 1987, pp. 3–10.
- [31] E. Catmull and R. Rom, “A class of local interpolating splines,” in *Computer aided geometric design*. Elsevier, 1974, pp. 317–326.
- [32] S. Sueda, “Catmull-rom splines,” 2016, material de curso (CSC471), Texas A&M University. Accedido: 15 de junio de 2026. [Online]. Available: <https://people.engr.tamu.edu/sueda/courses/CSC471/2016S/demos/alling/index.html>
- [33] J. Van Den Berg, S. J. Guy, M. Lin, and D. Manocha, “Reciprocal n-body collision avoidance,” in *Robotics research: the 14th international symposium ISRR*. Springer, 2011, pp. 3–19.
- [34] Microsoft, “Object.GetHashCode method,” 2023, accedido: 15 de junio de 2026. [Online]. Available: <https://learn.microsoft.com/en-us/dotnet/api/system.object.GetHashCode>

- [35] G. Fowler, L. C. Noll, and P. Vo, “Fowler-noll-vo hash function,” 1991, accedido: 15 de junio de 2026. [Online]. Available: <http://www.isthe.com/chongo/tech/comp/fnv/>
- [36] Mercuna, “Mercuna 3d navigation,” 2026, consultado el 15 de junio de 2026. [Online]. Available: <https://mercuna.com/>
- [37] Epic Games, “Unreal engine,” <https://www.unrealengine.com/>, 2026, consultado el 15 de junio de 2026.
- [38] Mercuna, “Mercuna 3d navigation licensing,” 2026, consultado el 15 de junio de 2026. [Online]. Available: <https://mercuna.com/licensing/>
- [39] Mercuna Developments Ltd., “Any surface navigation UE user guide,” <https://mercuna.com/documentation/any-surface-navigation-ue-user-guide/>, consultado el 15 de junio de 2026.
- [40] Unity Technologies, “Unity Asset Store,” <https://assetstore.unity.com/>, 2026, consultado el 15 de junio de 2026.
- [41] Softbrix Studio, “Nav3D – unity asset store,” <https://assetstore.unity.com/packages/tools/behavior-ai/nav3d-100285>, consultado el 15 de junio de 2026.
- [42] Nav3D, “Nav3d,” 2026, consultado el 15 de junio de 2026. [Online]. Available: <https://nav3d.pro/>
- [43] —, “Nav3d | pathfinding & navigation for flying ai,” 2026, consultado el 15 de junio de 2026. [Online]. Available: <https://assetstore.unity.com/packages/tools/game-toolkits/nav3d-pathfinding-navigation-for-flying-ai-337874>
- [44] Infohazard Games, “Hypernav: 3d pathfinding and navigation,” 2026, consultado el 15 de junio de 2026. [Online]. Available: <https://infohazardgames.com/docs/Infohazard.HyperNav/html/>
- [45] Tec-Maid, “Spatial pathfinding - 3d a* algorithm,” 2026, consultado el 15 de junio de 2026. [Online]. Available: https://github.com/Tec-Maid/wiki/tree/master/Spatial_Pathfinding
- [46] Godot Engine Contributors, “Godot engine,” <https://godotengine.org/>, 2026, version [Insert Version Here].
- [47] T. Salmela, “Game development using the open-source godot game engine,” 2022.